



UNIVERSIDAD DE CARTAGENA
Facultad de Ingeniería

**RESOLUCIÓN DE PROBLEMAS PNS BASADOS EN P-GRAPHS MEDIANTE
MODELOS MILP EN JAVA**

¹ Andrés Cardenas Jimenez 010, ² José Herrera Gómez 001, ³ Jader Morales Naar 028

Facultad de Ingeniería
IV semestre, Ingeniería de Sistemas
^{1,2,3} *Estudiante, Ingeniería de Sistemas, Universidad de Cartagena, Colombia*

1. Introducción

Se desarrolló una aplicación orientada a objetos en Java (NetBeans) con el uso de Estructuras de Datos internas del IDE, que permite resolver problemas PNS basados en P-Graph, aplicando los principios de diseño POO, la construcción de una interfaz gráfica para Analítica de resultados obtenidos mediante optimización dada por la integración de Solvers de modelos MILP y algoritmos Branch And Bound. Este proyecto de programación fue acompañado con su diseño del dominio del problema a través de la herramienta de diagramación “StarUML”. A continuación, se explicará brevemente cómo se desarrolló el proyecto y su funcionamiento.

2. Diseño del Sistema

El sistema se diseñó bajo un enfoque arquitectónico desacoplado de cuatro capas independientes utilizando el patrón de diseño por capas (N-Tier Architecture) y los principios de la programación orientada a objetos (POO). Este desacoplamiento asegura que las modificaciones en los algoritmos matemáticos de optimización o en los componentes de la interfaz gráfica no alteren la integridad estructural ni la lógica del resto de la aplicación.

Las capas que componen el sistema se definen de la siguiente manera:

Capa de Interfaz: Desarrollada sobre la plataforma JavaFX. Es la encargada de capturar las interacciones del usuario, gestionar los eventos del sistema (como la selección y el lanzamiento de solucionadores) y renderizar dinámicamente gráficos de barras de rendimiento y tablas analíticas interactuando con los hilos del sistema operativo.

Capa de Modelo: Representa el corazón del dominio del problema. Almacena las abstracciones matemáticas inmutables del Process Graph (P-Graph) sin poseer ninguna dependencia de librerías externas de optimización o vistas visuales.

Capa de Optimización: Contiene la lógica de traducción polimórfica que transforma las estructuras de datos de la red en matrices dispersas y ecuaciones lineales aptas para ser procesadas por múltiples motores de resolución independientes (Gurobi, CPLEX, GLPK y Apache Commons Math).

Capa de Persistencia: Responsable del intercambio seguro de datos con el sistema de almacenamiento físico, proveyendo un mecanismo de análisis sintáctico (parsing) de entrada y de serialización estructurada de salida.

El ciclo de ejecución del programa sigue un flujo lineal estrictamente controlado:

Entrada y Validación: El usuario selecciona un archivo plano de extensión .txt. La capa de persistencia lo valida mediante un sistema de excepciones personalizadas para garantizar la integridad referencial de los datos.

Instanciación del Grafo: Si la sintaxis es correcta, se pueblan colecciones dinámicas indexadas dentro del objeto contenedor principal de la capa de modelo.

Resolución Multilingüe: Al accionar la ejecución, los componentes inmutables del modelo se inyectan en el optimizador seleccionado por el usuario, activando los puentes nativos y bindings de los solucionadores MILP.

Despliegue y Auditoría: Los resultados estandarizados se devuelven en un hilo seguro hacia la interfaz gráfica de JavaFX para actualizar la tabla comparativa y el gráfico de barras. Simultáneamente, el sistema realiza una auditoría numérica interna para contrastar la precisión de los solvers y vuelca los metadatos de rendimiento en un archivo de auditoría local en formato .json dentro del directorio /data.

3. Dominio del Problema y Modelado en StarUML

De este modo, se identificaron los siguientes paquetes con sus respectivas clases con atributos y métodos:

a) Interfaz:

- App → ejecuta y da inicio a los hilos de javaFX.
- VentanaPrincipal → contiene toda la lógica visual del proyecto.
- PanelAnalitica → parte de la interfaz gráfica que contiene la tabla y gráfica de análisis.
- FilaComparativa → clase que maneja la interacción comparativa entre cada solver en la tabla de análisis.

b) Modelo:

- ModeloPGraph → almacena la lista de materiales, unidades operativas y tasas de flujo del modelo.
- Material → clase con instancias inmutables
- UnidadOperativa → clase con instancias inmutables
- TasaFlujo → clase con instancias inmutables

c) Optimización:

- OptimizadorGurobi → clase que construye y resuelve el modelo matemático entero mixto (MILP) utilizando el motor comercial Gurobi.
- OptimizadorApache → clase que construye y resuelve el modelo utilizando la librería de código abierto Apache Commons Math, ejecutando una optimización lineal continua (LP).
- OptimizadorCplex → clase que construye y resuelve el modelo mediante el motor comercial IBM ILOG CPLEX, aprovechando su arquitectura interna nativa para buscar y almacenar múltiples soluciones óptimas.
- OptimizadorGlpk → clase que construye y resuelve el modelo con el motor matemático de código abierto GLPK, aplicando de forma manual recortes de exclusión matemática para forzar la búsqueda de múltiples soluciones.

- `ResultadoOptimizacion` → clase que encapsula y estructura todos los datos resultantes tras la ejecución de cualquier solver, sirviendo como puente de transporte de información hacia la interfaz gráfica.

Entre otros paquetes están “**Persistencia**”, donde se almacenan las clases para la cargar/exportación de los archivos .TXT y .JSON, respectivamente; y, los paquetes especiales que son importados para poder crear instancias de clase que permitan generar objetos o interacciones en el programa.

Relaciones de causalidad

- **Dependencia:** Si una clase importa o usa métodos de otra clase externa, genera este tipo de relación, aplicadas comúnmente en los paquetes “java.util.” y “java.io”.
- **Herencia:** Las clase “App” hereda de la clase *Application* propia de javafx para poder ejecutar sus hilos e iniciar su ejecución en Escena.
- Por otro lado, la clase “LectorExcepciones” hereda de la clase *Exception* propia de java.lang. para personalizar mensajes de error.

Por último, se aplicaron los principios de abstracción y encapsulamiento donde todos los atributos definidos en cada paquete se declararon como privados, exponiendo su acceso únicamente a través de métodos públicos, como los Getters y Setters, y la inicialización de cada objeto en el constructor.

4. Definición de la Estructura de Datos

La representación y manipulación interna de la topología P-Graph en memoria se diseñó utilizando colecciones dinámicas indexadas y estructuras inmutables nativas de Java, garantizando la eficiencia en las búsquedas y la integridad de los datos durante los procesos de optimización.

a. Abstracción del Grafo como Contenedor Global

El Process Graph se modela como un grafo de tipo bipartito orientado a través de la clase contenedora *ModeloPGraph*. Esta clase actúa como un administrador centralizado que agrupa la infraestructura mediante tres listas dinámicas lineales basadas en la interfaz *List* e implementadas con *ArrayList*:

- List de tipo *Material* materiales: Almacena el conjunto indexado de los nodos de recursos y productos.
- List de tipo *UnidadOperativa* unidadesOperativas: Agrupa las plantas físicas de transformación industrial.
- List de tipo *TasaFlujo* tasaFlujos: Funciona como la lista de aristas dirigidas que interconectan formalmente los materiales con las unidades operativas.

b. Registros Inmutables para los Componentes del Dominio

Para representar las propiedades y los parámetros fijos de las mallas del sistema, se implementó el constructor sintáctico moderno de Java conocido como registros (record). Esta declaración genera automáticamente campos privados y finales, asegurando que los atributos sean de solo lectura para evitar alteraciones accidentales durante el cómputo multihilo o la inyección matricial en los solvers:

- **Material:** Almacena el identificador único del recurso, su clasificación dentro de la frontera operacional (raw_material para materias primas, intermediate para flujos de balance interno y product para demandas finales) y el requerimiento de entrega del mercado exterior.
- **UnidadOperativa:** Modela las plantas operativas reteniendo sus restricciones de escala física y los parámetros económicos requeridos para compilar la función objetivo financiera.
- **TasaFlujo:** Estructura la arista conectora del grafo orientado. Guarda los identificadores alfanuméricos de los elementos de origen y destino junto con el factor o multiplicador técnico que rige la escala de transformación de masa.

c. Estructura de Transferencia de Resultados (DTO)

El intercambio de datos entre la capa de optimización y la interfaz gráfica de usuario se unificó mediante el registro ResultadoOptimizacion. Esta estructura funciona como un Objeto de Transferencia de Datos o DTO (Data Transfer Object) que compacta los resultados combinatorios calculados por cualquier solver bajo las siguientes variables físicas y numéricas:

- **String solverUsado:** Identifica el resolutor que generó la corrida.
- **double costoTotal:** Almacena el valor óptimo devuelto por la función objetivo.
- **double tiempoEjecucionMs:** Registra el rendimiento del algoritmo medido en milisegundos.
- **Map de tipo String, Double flujosActivos:** Estructura un mapa asociativo de tipo HashMap que vincula de manera dinámica el identificador de la unidad operativa con el nivel continuo de operación calculado, descartando mediante límites de tolerancia numérica los elementos inactivos de la red sintetizada.
- **String estadoTexto e int solucionesEncontradas:** Resguardan las banderas de convergencia de la matriz (OPTIMAL, INFEASIBLE) y el tamaño total del pool de soluciones.

5. Implementación del modelo MILP (instrucciones en el lenguaje)

La traducción programática de la topología P-Graph hacia un modelo de Programación Lineal Entera Mixta (MILP) se consolidó de forma abstracta y reutilizable dentro del paquete de optimización. A nivel general, el diseño del software permite mapear los componentes del grafo mediante las instrucciones y estructuras lógicas comunes que requiere cualquier resolutor matemático estándar.

a. Declaración Indexada de Variables de Decisión:

El algoritmo recorre las estructuras de datos inmutables del grafo para inicializar los vectores de decisión en el entorno del resolutor seleccionado, registrándose en tablas asociativas de tipo HashMap para su posterior manipulación:

- **Variables Discretas (Binarias):** Controlan la existencia o construcción física de las mallas de la red. Toman valores estrictos en el espacio entero $\{0, 1\}$ y representan si una planta se activa o se descarta en la topología final.
- **Variables Continuas:** Representan los flujos de masa y niveles de operación reales. Su rango se acota dinámicamente entre cero y el límite máximo de capacidad técnica definido por el problema.

- **Restricciones Lógicas de Activación:** En cualquier motor matemático se inyecta una inequación lineal que vincula ambos componentes. Esta instrucción establece que el nivel de operación continua de una unidad debe ser menor o igual a su capacidad máxima multiplicada por su variable binaria, impidiendo el paso de flujo a través de infraestructura inactiva.
- b. **Compilación Lineal de Balances de Masa:**

El principio fundamental de conservación de la masa se construye recorriendo los objetos de tipo Material y evaluando de manera anidada las aristas orientadas de la red. Para cada nodo se genera una expresión polinomial lineal donde las tasas técnicas de flujo se añaden acumulativamente según el sentido del vector:

 - **Flujos de Entrada (Producción):** Si el material es el destino del flujo, se añade a la ecuación el coeficiente técnico de escala con signo positivo.
 - **Flujos de Salida (Consumo):** Si el material actúa como fuente o suministro de una planta, el término se inyecta con signo negativo.
 - **Condiciones de Frontera de la Red:** Una vez estructurado el polinomio para cada nodo, el sistema aplica las restricciones algebraicas del resolutor según el rol del elemento: los productos finales se igualan o restringen mediante cotas de tipo mayor o igual para satisfacer la demanda mínima del mercado, los materiales intermedios se fuerzan a una igualdad estricta a cero para garantizar un balance neto sin fugas ni acumulaciones, y las materias primas se omiten de este balance por actuar como fuentes libres de suministro entrante.
- c. **Formulación de la Función Objetivo Global:**

La optimización económica del sistema unifica los costos fijos y variables de toda la infraestructura bajo una única directiva de minimización. Utilizando las clases constructoras de expresiones del resolutor, se define una sumatoria que multiplica el costo fijo de cada planta por su variable binaria de activación, y el costo proporcional por su variable continua de operación. Finalmente, se invoca la rutina de ejecución central del motor matemático para procesar las matrices resultantes mediante algoritmos de optimización combinatoria.
- d. **Extracción Homogénea de Soluciones:**

Tras la convergencia de los algoritmos de resolución, el sistema evalúa los indicadores de éxito y el estado de la búsqueda. Para los motores que admiten configuraciones multi-escenario, el entorno recupera de manera indexada tanto la solución óptima absoluta como las combinaciones subóptimas válidas almacenadas en memoria. Todos los flujos activos que superan el umbral de tolerancia de redondeo numérico del sistema se leen secuencialmente y se encapsulan junto a sus metadatos de rendimiento en registros estandarizados de transferencia de datos para alimentar los componentes gráficos de la interfaz de usuario.

6. Integración inicial de solvers

Entre los solver integrados se usaron “Gurobi v1302” y “Apache Commons Math 4 Core” como optimizadores obligatorios, y como solvers adicionales se escogieron “GLPK v4.65” y “CPLEX v222”.

La integración de estos archivos se realizó mediante APIs Java y la ejecución por línea de comandos, como métodos tradicionales y sencillos de lograr en el funcionamiento óptimo. Para cada optimizador/solver se necesitaron integrar los archivos comprimidos .JAR de cada uno como librerías dentro de **classpath** y **modulepath** del proyecto con ANT, estos archivos se almacenaron en una librería llamada “*lib*” creada dentro de los archivos del proyecto a manera de **ruta relativa** para una rápida ubicación de los archivos.

a) Integración de Apache Commons Math:

Es necesario conseguir los archivos de recursos .JAR en la página oficial de Apache Commons Math, para ello se necesita descargar y descomprimir el archivo “commons-math4-4.0-beta1-bin.zip”, el cual nos dará varios de los archivos que necesitamos.

En la carpeta “*lib*”, utilizada como ruta relativa, almacenamos los 6 archivos obtenidos en la subcarpeta **classpath**, adicionalmente, necesitaremos el archivo “commons-numbers-core-1.3.jar” que no viene incluido dentro del directorio “.zip” para lograr la integración de las variables lineales en el modelo creado en el solver internamente desde java.

b) Integración de Gurobi:

Desde la página oficial de Gurobi, se descargó el kit de instalación del solver el cual incluye los archivos binarios “.dll” y archivos “.jar”, en dicha instalación se siguieron los ajustes predeterminados para evitar problemas de uso.

Necesariamente, el solver solicita tener una licencia habilitada para poder acceder a sus servicios, para ello se accedió a la licencia académica gratuita que otorga y se anexó a la ruta absoluta del usuario del sistema operativo C:\Users\User para que pueda ser reconocida automáticamente por el Optimizador al momento de su arranque debido a que dicho software verifica por defecto esa ruta.

Después de realizar dicho procedimiento, se adicionan los archivos “.jar” en la ruta relativa de las librerías para **classpath** previamente mencionadas

c) Integración de CPLEX:

Para este solver, se requiere acceder al portal oficial de IBM y descargar el instalador de IBM ILOG CPLEX Optimization Studio. Al igual que en casos anteriores, es posible acceder a una versión completa sin restricciones mediante la iniciativa académica de IBM. Durante el proceso, se siguieron los ajustes y rutas de instalación predeterminados para evitar problemas de configuración y asegurar que la licencia interna se vincule correctamente.

Una vez finalizada la instalación, es necesario explorar el directorio local donde se instaló el software para ubicar el archivo “cplex.jar” (típicamente localizado dentro del subdirectorio cplex\lib). Este archivo se extrae y se adiciona a nuestra carpeta “*lib*” como ruta relativa para el classpath. Finalmente, es indispensable vincular los archivos binarios “.dll” del programa; para ello, se debe tomar la ruta de la carpeta bin de CPLEX y agregarla a la variable de

entorno PATH del sistema operativo para que pueda ser reconocida automáticamente al momento de la ejecución.

d) Integración de GLPK:

Desde la página web oficial del proyecto GLPK (específicamente a través de su interfaz GLPK para Java), es necesario descargar los archivos de la distribución correspondiente a la arquitectura del sistema operativo. Al descomprimir el archivo descargado, se obtendrán tanto las librerías dinámicas como el archivo empaquetado de Java necesarios para su funcionamiento.

El archivo “glpk-java.jar” (o su equivalente según la versión descargada) se debe almacenar en la carpeta “lib” para ser añadido a la ruta relativa del classpath. Adicionalmente, al ser un solver escrito en C, es estrictamente necesario asegurar que la ruta donde se encuentra el archivo de la librería nativa (el archivo “.dll” en Windows o “.so” en Linux) esté configurada en las variables de entorno del sistema operativo, de modo que el modelo creado en Java pueda comunicarse internamente con el motor del solver.

e) Integración de JavaFX:

Es necesario dirigirse a la página web oficial, para mayor solución: Gluon, para descargar el kit de desarrollo, conocido como JavaFX SDK, correspondiente a la arquitectura y sistema operativo en uso. Una vez descargado y descomprimido el archivo “.zip”, obtendremos los binarios y librerías fundamentales para la construcción de la interfaz gráfica.

De la carpeta extraída, tomamos los archivos “.jar” necesarios (tales como javafx.controls.jar, javafx.fxml.jar, javafx.graphics.jar, entre otros) y los almacenamos en nuestra carpeta “lib”, utilizada como ruta relativa, para añadirlos al **classpath** del proyecto de la misma manera que en los casos anteriores.

Adicionalmente, dado que JavaFX fue desacoplado de las versiones modernas del JDK estándar, el entorno solicita una configuración especial al momento de su arranque. Es estrictamente necesario modificar los argumentos de la Máquina Virtual de Java (VM options) en el entorno de ejecución, indicando la ruta hacia las librerías con el parámetro **modulepath** e incluyendo el comando --add-modules con los módulos específicos utilizados en el código.

f) Integración de los solvers en la VM JAVA:

Finalmente, para la integración funcional de cada uno de los solvers a excepción de Apache, se requiere indicar a la Máquina Virtual Java del proyecto donde están guardados los archivos binarios “.dll” para que los solver Gurobi, Glpk y Cplex, incluso para JavaFX, puedan reconocer sus respectivos archivos de ejecución. Este lugar corresponde a la ruta donde se hospedaron los archivos creados durante el proceso de instalación de los solvers.

Esto se hace mediante la integración del siguiente comando (si se siguieron las configuraciones predeterminadas):

```
--enable-native-access=ALL-UNNAMED --add-modules
javafx.controls,javafx.graphics,javafx.base
-Djava.library.path="C:\gurobi1302\win64\bin;C:\glpk-4.65\w64;C:\Program
Files\IBM\ILOG\CPLEX_Studio_Community222\cplex\bin\x64_win64;C:\javafx-sdk-2
6.0.1\bin" -Dfile.encoding=UTF-8
```

Para insertar este comando y guardar los cambios, se debe seguir la siguiente ruta:

Propiedades del proyecto --> Run --> VM Options:

Con este comando, los solver estarán listos para inicializarse.

7. Persistencia básica

La capa de persistencia se encarga de gestionar de forma controlada el ciclo de lectura de los archivos de entrada que definen la topología de la red y la exportación estructurada de los resultados numéricos obtenidos por los solucionadores.

a. Lectura de Modelos (.TXT)

Desarrollada en la clase CargarPGraph mediante una máquina de estados basada en el tipo enumerado Seccion (MATERIALES, UNIDADES_OPERATIVAS, TASA_FLUJOS). El componente procesa el archivo línea por línea omitiendo comentarios y líneas vacías, y utiliza expresiones regulares con las clases Pattern y Matcher para segmentar e interpretar las propiedades de los materiales, los límites de capacidad, los costos fijos y proporcionales de las unidades, y las tasas de flujo asociadas.

b. Control de Excepciones Especializadas

En caso de detectar anomalías lógicas o estructurales durante el parseo de datos, la lectura se interrumpe de forma controlada mediante la clase LectorExcepciones, la cual extiende de Exception. Esta excepción personalizada captura y propaga el número exacto de línea y el motivo semántico del fallo sintáctico, evitando que la aplicación sufra cierres inesperados.

c. Exportación de Resultados (.JSON)

La clase ExportarSoluciones automatiza la serialización de los objetos de tipo ResultadoOptimizacion estructurándolos en formato JSON a través de un flujo BufferedWriter. El archivo final organiza la información discriminando un bloque para los metadatos del entorno (solver utilizado, tiempo de ejecución y estado de la solución) y otro bloque específico para las variables y flujos físicos que resultaron activos durante el proceso de optimización combinatoria.

8. Bibliografía

<https://commons.apache.org/proper/commons-math/>

<https://www.gnu.org/software/glpk/>

<https://highs.dev/>

<https://www.scipopt.org/>

<https://www.gurobi.com/>

<https://www.ibm.com/es-es/products/ilog-cplex-optimization-studio>

9. Anexos

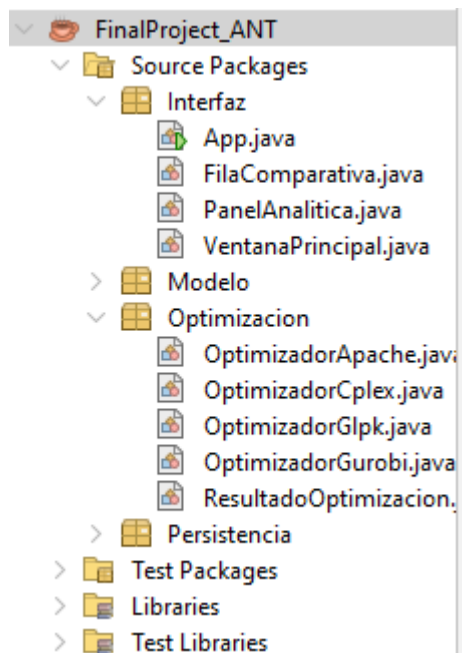


Figura 1. Distribución de paquetes Java

```
// CARDENAS_ANDRES_010_HERRERA_JOSE_001_MORALES_JADER_028
package Interfaz;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class App extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Instanciamos nuestra vista principal
        VentanaPrincipal vista = new VentanaPrincipal(primaryStage);

        // Configuración de escena (tamaño de la ventana)
        Scene scene = new Scene(vista.getRoot(), 1000, 600);

        primaryStage.setTitle("Optimizador PNS - MILP Solvers");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args); // Arranca el hilo de JavaFX
    }
}
```

Figura 2. Clase App

